

5 分钟启动Quick Start	配置 & 操作application.yml	Chat API 示例Controller	RAG 入库 & 问答Retrieval
创建项目 <pre>curl https://start.spring.io/starter.zip \ -d bootVersion=4.0.0 -d javaVersion=25 \ -d dependencies=web,actuator,validation \ -d name=spring-ai -o java-ai.zip</pre> 最小依赖 <pre><dependency> <groupId>org.springframework.ai</groupId> <artifactId>spring-ai-starter-model-openai</artifactId> </dependency> <dependency> <groupId>org.springframework.ai</groupId> <artifactId>spring-ai-starter-vector-store-pgvector</artifactId> </dependency></pre> JDK 25 当前 LTS;生产用 LTS + 锁定镜像 ./mvnw spring-boot:run 本地启动;key 走环境变量 curl :8080/chat?q=... 第一条链路先跑通再扩 RAG/工具	spring: ai: openai: api-key: \${OPENAI_API_KEY} chat.options.model: gpt-4.1-mini embedding.options.model: text-embedding-3-small vectorstore.pgvector: initialize-schema: true dimensions: 1536 threads.virtual.enabled: true management.endpoints.web.exposure.include: health,metrics,prometheus 本地向量库 docker run --name pgvector -p 5432:5432 \ -e POSTGRES_PASSWORD=postgres \ -e POSTGRES_DB=spring-ai-rag \ ankane/pgvector OPENAI_API_KEY 只放 env / secret,绝不提交 timeout/retry 所有模型调用必须设上限 Actuator health + metrics + prometheus	@RestController class ChatApi { private final ChatClient chat; ChatApi(ChatClient.Builder b) { this.chat = b.defaultSystem("你是严谨的 Java 助手; 回答要短, 给代码优先。").build(); } @GetMapping("/{chat}") String chat(@RequestParam String q) { return chat.prompt().user(q).call().content(); } } 结构化输出 record Todo(String title, int priority) {} Todo t = chat.prompt() .user("'把'明早修登录bug'转成任务'") .call().entity(Todo.class); .chatResponse() 拿 token usage / metadata / finish reason .stream() SSE 流式输出;长回答体验更好	@PostMapping("/ingest") void ingest(@RequestBody String text) { var docs = List.of(new Document(text, Map.of("tenant", "demo", "source", "note"))); vectorStore.add(tokenSplitter.split(docs)); } @GetMapping("/ask") String ask(String q) { var advisor = QuestionAnswerAdvisor.builder(vectorStore) .searchRequest(SearchRequest.builder() .topK(5).similarityThreshold(0.72).build()) .build(); return chat.prompt().user(q) .advisors(advisor).call().content(); } metadata tenant/user/ACL/source/page 必存 topK + threshold 先召回够,再防低质上下文污染 citation 把 source/page/uri 带响应
工具调用Tool Calling	LangChain4j 示例Alt Stack	MCP / 本地 / 部署Ops	工程清单Checklist
@Service class OrderTools { @Tool(description="按订单号查询物流状态") String track(@ToolParam(description="订单号") String id) { return orderRepo.findStatus(id); // 服务端再验权限 } } @GetMapping("/agent") String agent(String q, OrderTools tools) { return chat.prompt() .user(q) .tools(tools) .call().content(); } @Tool 方法名/描述影响模型是否会调用 @ToolParam 参数越清楚,越少乱填 最小授权 每次请求只挂必要工具 服务端校验 权限、金额、路径、SQL 都不能信模型 审计日志 记录 user、tool、args、result、traceld	interface Assistant { @SystemMessage("你是 Java 架构师, 回答要可执行") String chat(@MemoryId String session, @UserMessage String message); } Assistant bot = AiServices.builder(Assistant.class) .chatLanguageModel(model) .chatMemoryProvider(id -> MessageWindowChatMemory.withMaxMessages(20)) .tools(new OrderTools()) .build(); AiServices 接口即 AI 服务,适合业务层注入 ChatMemory 短期会话窗口,不是永久知识库 RetrievalAugmentor RAG 插入点;可自定义 query/router Quarkus 启动快/Native 友好,轻量服务可选 Spring AI Boot 生态优先;企业集成更顺	Ollama 本地开发 ollama pull llama3.1 spring.ai.ollama.chat.options.model=llama3.1 spring.ai.ollama.base-url=http://localhost:11434 MCP 接入思路 MCP Server: tools/resources/prompts Spring AI Client -> ToolCallbackProvider ChatClient.prompt().tools(mcpTools).call() Stdio 本地工具进程;适合文件/命令类能力 SSE / HTTP 远端 MCP 服务;需鉴权与租户隔离 Docker JVM 镜像设内存、时区、非 root Buildpacks ./mvnw spring-boot:build-image Cost Meter 记录 tokens、模型、租户、traceld	record DTO validationRAG ACL citations timeout retry rate limittools audit golden set metricsrollback 入门顺序 · chat → stream → structured output → RAG → tools → eval 业务边界 · prompt 只做表达;规则/权限/金额在 Java service RAG 权限 · tenant/user/ACL 必须检索前过滤,不是生成后遮挡 工具风险 · 写操作、付款、删数据必须人工确认或双重校验 可观测 · prompt 版本、tokens、latency、tool trace、source docs 评测 · golden set + 回归测试;升级模型/提示/索引都要跑 并发 · virtual threads 能抗阻塞 IO,但模型 API 仍要限流 缓存 · embedding/检索/稳定提示可缓存;隐私数据设 TTL 上线 · 模型、提示、向量索引都版本化,支持灰度与回滚 排错 · 先看 key/model/timeout,再看 prompt、RAG 命中文档、tool args